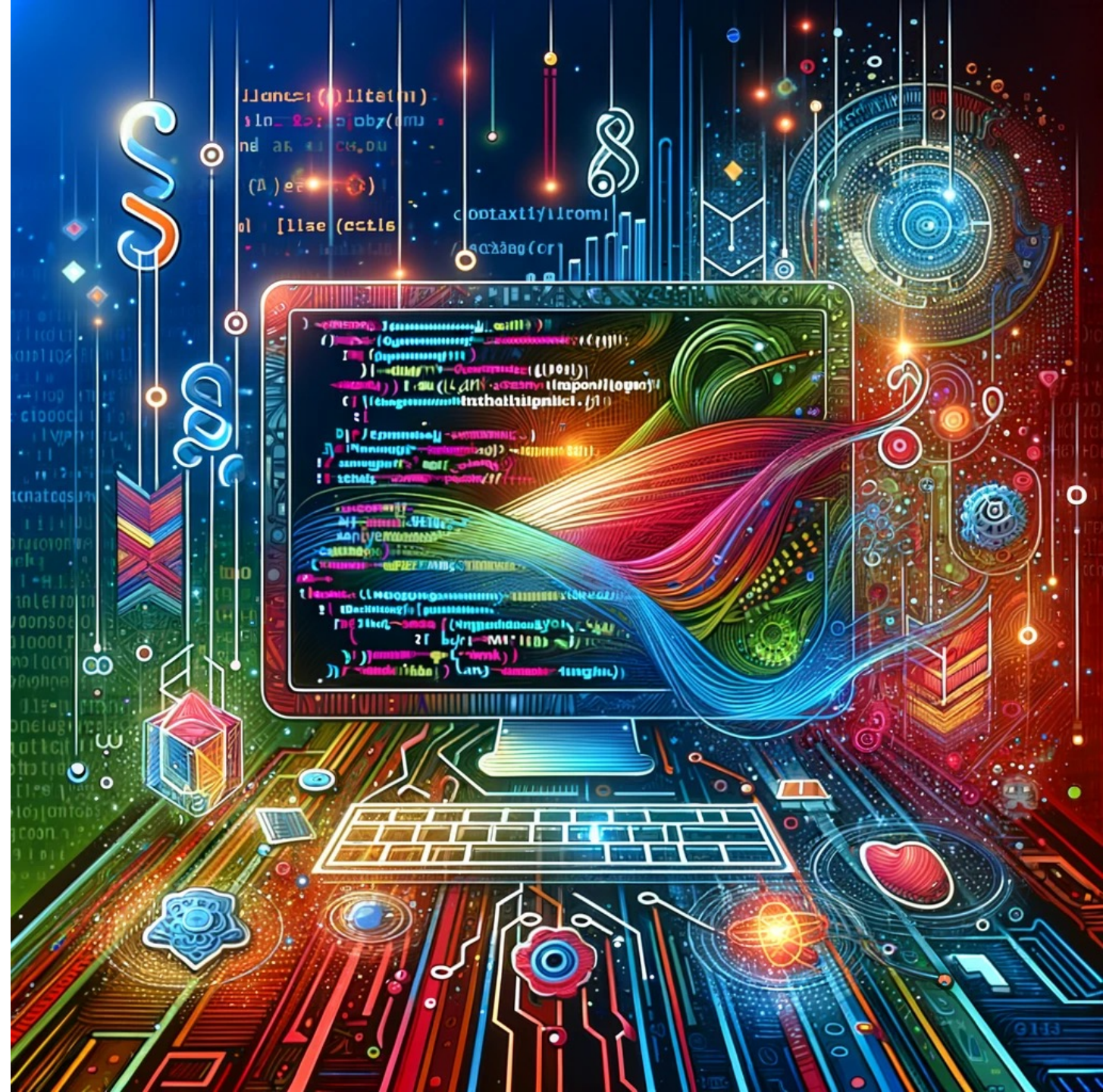


BİLGİSAYAR PROGRAMLAMA





Bubble Sort'un Pointer (call-by-reference mantığı) Kullanılarak Uygulanması

10 elemanlı bir tam sayı dizisi için pointer kullanarak **Bubble Sort algoritmasını** gerçekleştiren bir C programı yazınız.

- Dizi elemanlarını önce **orijinal sırayla** ekrana yazdırınız.
- Diziyi sıralamak için **bubbleSort** adlı bir fonksiyon kullanınız.
- İki elemanın yerini değiştirmek için **swap** adlı ayrı bir fonksiyon yazınız. **swap fonksiyonu**, değiştirilecek dizi elemanların **adreslerini (& operatörü kullanarak)** parametre olarak almalıdır.
- C dilinde fonksiyonlara parametrelerin varsayılan olarak **call-by-value** ile gönderildiğini göz önünde bulundurarak, pointer ve * operatörünü kullanarak elemanların yerini değiştiriniz.
- Sıralama tamamlandıktan sonra diziyi **sıralı şekilde** ekrana yazdırınız.

```
1  /* Fig. 7.15: fig07_15.c
2     Bu program, bir diziye değerler atar, Bubble Sort algoritması ile sıralar
3     ve sonuçları ekrana yazdırır.
4  */
5
6  #include <stdio.h>
7  #define SIZE 10      // Dizi boyutunu sabit bir değer olarak tanımlıyoruz
8
9  // Bubble Sort fonksiyonunun prototipi
10 void bubbleSort(int *, const int);
11
12 int main() {
13     // 10 elemanlı bir dizi tanımlanıyor ve başlangıç değerleri atanıyor
14     int a[SIZE] = {2, 6, 4, 8, 10, 12, 89, 68, 45, 37};
15     int i;
16
17     // Dizinin sıralanmamış halini ekrana yazdırıyoruz
18     printf("Data items in original order\n");
19     for (i = 0; i < SIZE; i++)
20         printf("%4d", a[i]); // Her elemanı ekrana yazdır
21
22     // Bubble Sort fonksiyonunu çağırarak diziyi sıralıyoruz
23     bubbleSort(a, SIZE);
24
25     // Dizinin sıralı halini ekrana yazdırıyoruz
26     printf("\nData items in ascending order\n");
27     for (i = 0; i < SIZE; i++)
28         printf("%4d", a[i]);
29
30     printf("\n"); // Yeni satır ekleyerek programın çıktısını düzenli hale getiriyoruz
31
32     return 0; // Programın başarılı şekilde bittiğini belirtiyoruz
33 }
```

```
35  /*
36     bubbleSort() fonksiyonu, pointer kullanarak verilen diziyi sıralar.
37     - array: Sıralanacak dizinin pointer adresi
38     - size: Dizinin eleman sayısı
39  */
40  void bubbleSort(int *array, const int size) {
41      void swap(int *, int *); // swap fonksiyonunun prototipi
42      int pass, j;
43
44      // Bubble Sort işlemi: Diziyi tarayarak elemanları sıralar
45      for (pass = 0; pass < size - 1; pass++) { // Kaç tur sıralama yapılacağını belirler
46          for (j = 0; j < size - 1; j++) { // Her turda diziyi baştan sona tarar
47              if (array[j] > array[j + 1]) // Eğer soldaki eleman sağdakinden büyükse
48                  swap(&array[j], &array[j + 1]); // İki elemanın yerini değiştir
49          }
50      }
51  }
52
53  /*
54     swap() fonksiyonu, iki elemanın yerini değiştirir.
55     - element1Ptr: Değiştirilecek birinci elemanın adresi
56     - element2Ptr: Değiştirilecek ikinci elemanın adresi
57  */
58  void swap(int *element1Ptr, int *element2Ptr) {
59      int hold = *element1Ptr; // İlk elemanın değerini geçici değişkene sakla
60      *element1Ptr = *element2Ptr; // İlk eleman, ikinci elemanın değerini alır
61      *element2Ptr = hold; // İkinci eleman, ilk elemanın değerini alır (değişim tamamlandı)
62  }
```



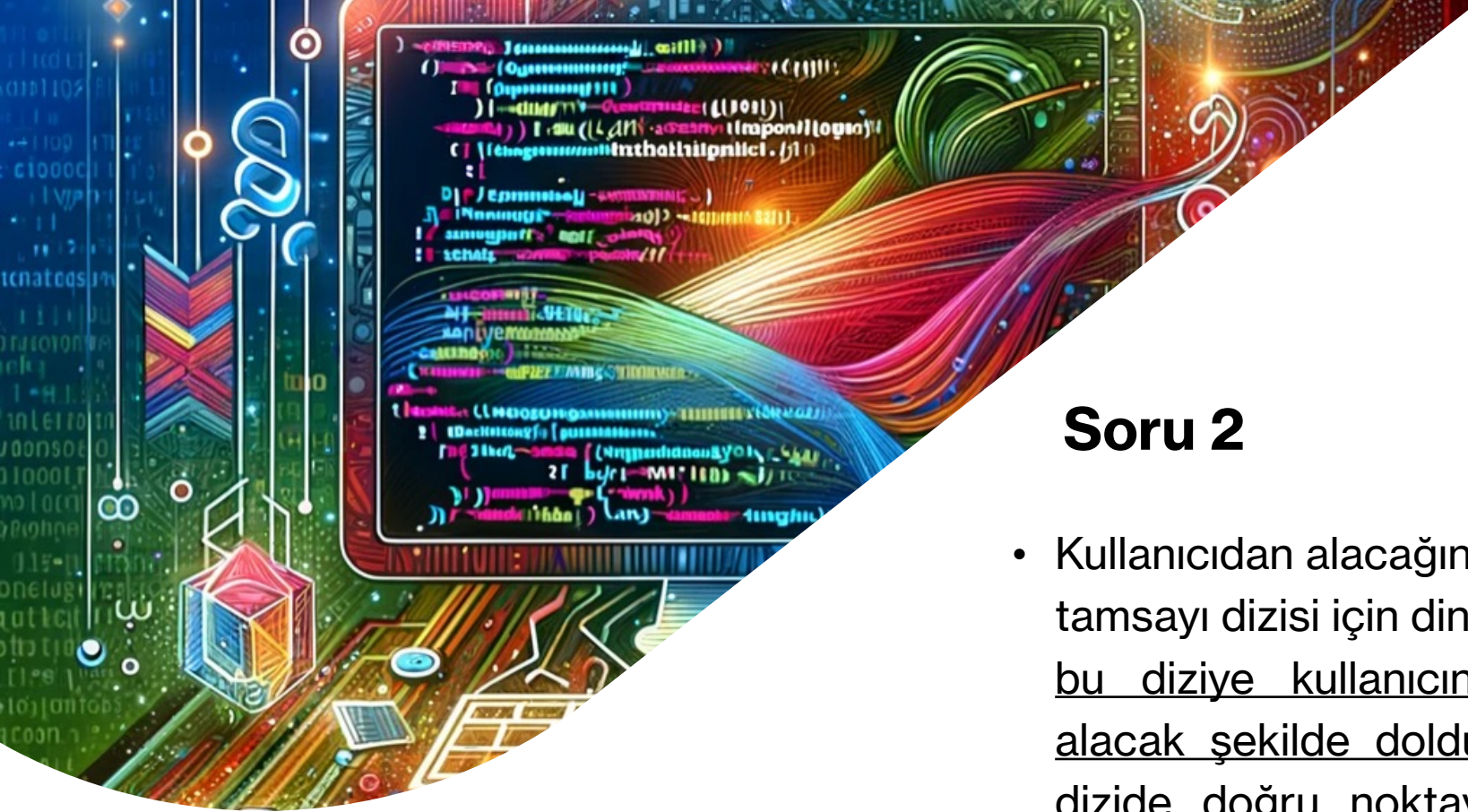
Soru 1

- Kullanıcıdan alacağınız bir sayı (N) büyüklüğünde bir tamsayı dizisi için dinamik bellek ayırsın. Daha sonra bu diziyi kullanıcının gireceği N adet tamsayı ile doldurun. Son olarak, doldurulan dizinin elemanlarını ve dizinin toplam eleman sayısını ekrana yazdırın. Programın sonunda ise ayrılan dinamik belleği serbest bırakın.

```
1  #include <stdio.h>
2  #include <stdlib.h> // malloc ve free fonksiyonlarını kullanabilmek için gerekli
3
4  int main() {
5
6      int N, i;        // N → dizinin boyutu, i → döngü değişkeni
7      int *dizi;       // Dinamik olarak oluşturulacak diziyi gösterecek pointer
8
9      // Kullanıcıdan dizinin boyutunu alma
10     printf("Dizi boyutunu girin: ");
11     scanf("%d", &N);
12
13     // -----
14     // Dinamik bellek ayırma işlemi
15     // -----
16     // malloc fonksiyonu heap bölgesinden bellek ayırır
17     // N tane int saklayacak kadar yer ayırıyoruz
18     dizi = (int *)malloc(N * sizeof(int));
19
20     // Eğer bellek ayrılmazsa malloc NULL döndürür
21     if (dizi == NULL) {
22         printf("Bellek ayırma hatası!\n");
23         return -1; // Programı hata ile sonlandır
24     }
25
26     // Kullanıcıdan dizi elemanlarının alınması
27     printf("%d tane sayı girin:\n", N);
28
29     for (i = 0; i < N; i++) {
30         scanf("%d", &dizi[i]);
31     }
32
33
```

```
34     // Dizinin elemanlarının ekrana yazdırılması
35     printf("Dizinin elemanlari:\n");
36
37     for (i = 0; i < N; i++) {
38         printf("%d ", dizi[i]);
39     }
40
41     // Dizinin toplam eleman sayısını ekrana yazdırma
42     printf("\nDizi boyutu: %d\n", N);
43
44     // -----
45     // Dinamik olarak ayrılan belleği serbest bırakma
46     // -----
47     // free() ile işaretçi için ayrılan bellek alanı geri verilir.
48     free(dizi);
49
50     return 0;
51 }
```

```
Dizi boyutunu girin: 10
10 tane sayı girin:
1
2
3
4
5
6
7
8
9
10
Dizinin elemanlari:
1 2 3 4 5 6 7 8 9 10
Dizi boyutu: 10
```



Soru 2

- Kullanıcıdan alacağınız bir sayı (N) büyüklüğünde bir tamsayı dizisi için dinamik bellek ayırsın. Daha sonra bu diziye kullanıcının gireceği tamsayıları sıralı alacak şekilde doldurun. Yani girilen her tamsayı dizide doğru noktaya yerleştirilir ve kullanıcıdan eleman alma işlemi dizinin son elemanı da girilene kadar devam eder. Programın sonunda ise ayrılan dinamik belleği serbest bırakın.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Bu fonksiyon girilen sayıyı dizide doğru sıraya yerleştirir
5  void insertInOrder(int *array, int num, int index) {
6
7      // Yeni elemanın yerleştirileceği konumu bulmak için
8      // sondan başlayarak sola doğru ilerliyoruz
9      int i = index - 1;
10
11     // Eğer soldaki eleman num'dan büyükse
12     // elemanı bir sağa kaydırıyoruz
13     while (i >= 0 && array[i] > num) {
14         array[i + 1] = array[i]; // elemanı sağa kaydır
15         i--;                     // bir sonraki elemana geç
16     }
17
18     // Doğru konum bulunduğunda sayıyı yerleştir
19     array[i + 1] = num;
20 }
21
22 int main() {
23
24     int N, num, i;
25
26     // Kullanıcıdan kaç sayı gireceğini alıyoruz
27     printf("Toplam kac sayi girilecek: ");
28     scanf("%d", &N);
29
30     // Dinamik bellek ayırma
31     int *array = (int *)malloc(N * sizeof(int));

```

```

33 // Eğer bellek ayrılamazsa hata mesajı ver
34 if (array == NULL) {
35     printf("Bellek tahsis edilemedi.\n");
36     return -1;
37 }
38
39 // Kullanıcıdan sayıları alıyoruz
40 // Her sayı girildiğinde doğru konuma yerleştiriliyor
41 for (i = 0; i < N; i++) {
42
43     printf("%d. sayiyi giriniz: ", i + 1);
44     scanf("%d", &num);
45
46     // Girilen sayıyı dizide doğru sıraya yerleştir
47     insertInOrder(array, num, i);
48 }
49
50 // Son durumda dizi zaten sıralı olduğu için
51 // doğrudan yazdırıyoruz
52 printf("Siralanan dizi: ");
53
54 for (i = 0; i < N; i++) {
55     printf("%d ", array[i]);
56 }
57
58 // Dinamik olarak ayrılan belleği serbest bırakıyoruz
59 free(array);
60
61 return 0;
62 }

```

```

Toplam kac sayi girilecek: 5
1. sayiyi giriniz: 12
2. sayiyi giriniz: 9
3. sayiyi giriniz: -3
4. sayiyi giriniz: 45
5. sayiyi giriniz: 0
Siralanan dizi: -3 0 9 12 45

```



Soru 3

- Kullanıcıdan alacağınız satır (M) ve sütun (N) sayılarına göre bir tamsayı matrisi (iki boyutlu dizi) için dinamik bellek ayırın. Daha sonra bu matrisi kullanıcının gireceği değerlerle doldurun. Matrisin her bir elemanı için kullanıcıdan ayrı ayrı değer alınacaktır. Son olarak, doldurulan matrisin elemanlarını ekrana yazdırın. Programın sonunda ise ayrılan dinamik belleği serbest bırakın.

```

1  #include <stdio.h>
2  #include <stdlib.h> // malloc ve free fonksiyonları için gerekli
3
4  int main() {
5      int M, N;        // M = satır sayısı, N = sütun sayısı
6      int i, j;        // Döngü değişkenleri
7      int **matris;    // İki boyutlu dinamik dizi için pointer to pointer
8
9      // Kullanıcıdan matrisin satır ve sütun sayısını al
10     printf("Matrisin satir sayisini girin: ");
11     scanf("%d", &M);
12     printf("Matrisin sutun sayisini girin: ");
13     scanf("%d", &N);
14
15     // 1. Adım: Satırlar için bellek ayırma
16     // Burada M tane satır adresi tutacak kadar yer ayırıyoruz
17     matris = (int **)malloc(M * sizeof(int *));
18
19     // Eğer bellek ayrılamazsa hata verip programı sonlandır
20     if (matris == NULL) {
21         printf("Bellek ayirma hatasi!\n");
22         return -1;
23     }
24
25     // 2. Adım: Her satır için sütun belleği ayırma
26     // Her satır için N tane int saklayacak kadar yer ayırıyoruz
27     for (i = 0; i < M; i++) {
28         matris[i] = (int *)malloc(N * sizeof(int));
29
30         // Eğer herhangi bir satır için bellek ayrılamazsa
31         // daha önce ayrılan bellekleri serbest bırakıp çıkıyoruz
32         if (matris[i] == NULL) {
33             printf("Bellek ayirma hatasi!\n");
34
35             for (j = 0; j < i; j++) {
36                 free(matris[j]);
37             }
38             free(matris);
39
40             return -1;
41         }
42     }

```

```

44
45     // 3. Adım: Matris elemanlarını kullanıcıdan alma
46     printf("Matrisin elemanlarini girin:\n");
47
48     for (i = 0; i < M; i++) {
49         for (j = 0; j < N; j++) {
50             printf("Eleman [%d][%d]: ", i, j);
51             scanf("%d", &matris[i][j]);
52         }
53     }
54
55     // 4. Adım: Matrisi ekrana yazdırma
56     printf("Matris:\n");
57
58     for (i = 0; i < M; i++) {
59         for (j = 0; j < N; j++) {
60             printf("%d ", matris[i][j]);
61         }
62         printf("\n"); // Her satırdan sonra alt satıra geç
63     }
64
65     // 5. Adım: Ayrılan belleği serbest bırakma
66     // Önce her satır için ayrılan bellek serbest bırakılır
67     for (i = 0; i < M; i++) {
68         free(matris[i]);
69     }
70
71     // En son satır adreslerini tutan ana pointer serbest bırakılır
72     free(matris);
73
74     return 0;

```

```

Matrisin satir sayisini girin: 3
Matrisin sutun sayisini girin: 4
Matrisin elemanlarini girin:
Eleman [0][0]: 1
Eleman [0][1]: 4
Eleman [0][2]: 5
Eleman [0][3]: 6
Eleman [1][0]: 7
Eleman [1][1]: 8
Eleman [1][2]: 9
Eleman [1][3]: 0
Eleman [2][0]: -1
Eleman [2][1]: 4
Eleman [2][2]: 0
Eleman [2][3]: -7
Matris:
1 4 5 6
7 8 9 0
-1 4 0 -7

```



Soru 4

- Kullanıcıdan alacağınız satır (R) ve sütun (C) sayılarına göre bir tamsayı matrisi (iki boyutlu dizi) için dinamik bellek ayırın. Bu matrisi kullanıcının gireceği değerlerle doldurun. Daha sonra, bu matrisin transpozunu (yani satırların sütun, sütunların satır olarak yer değiştirdiği yeni bir matris) alıp, hem orijinal matrisi hem de transpozunu ekrana yazdırın. Programın sonunda ise ayrılan dinamik belleği serbest bırakın.

```

1  #include <stdio.h>
2  #include <stdlib.h> // malloc ve free için gerekli
3
4  int main() {
5      int R, C;           // R = satır sayısı, C = sütun sayısı
6      int i, j;           // Döngü değişkenleri
7      int **matris, **transpoz; // Orijinal matris ve transpoz matris için pointerlar
8
9      // Kullanıcıdan matrisin satır ve sütun sayılarını al
10     printf("Matrisin satir sayisini girin: ");
11     scanf("%d", &R);
12     printf("Matrisin sutun sayisini girin: ");
13     scanf("%d", &C);
14
15
16     // ORIJİNAL MATRİS İÇİN DİNAMİK BELLEK AYIRMA (R x C)
17     // Önce R tane satır adresi tutacak yer ayırıyoruz
18     matris = (int **)malloc(R * sizeof(int *));
19
20     // Her satır için C tane int saklayacak alan ayırıyoruz
21     for (i = 0; i < R; i++) {
22         matris[i] = (int *)malloc(C * sizeof(int));
23     }
24
25     // TRANSPOZ MATRİS İÇİN DİNAMİK BELLEK AYIRMA (C x R)
26     // Önce C tane satır adresi için yer ayırıyoruz
27     transpoz = (int **)malloc(C * sizeof(int *));
28
29     // Her satır için R tane int saklayacak alan ayırıyoruz
30     for (i = 0; i < C; i++) {
31         transpoz[i] = (int *)malloc(R * sizeof(int));
32     }
33
34     // ORIJİNAL MATRİS ELEMANLARINI KULLANICIDAN ALMA
35     printf("Matris elemanlarini girin:\n");
36
37     for (i = 0; i < R; i++) {
38         for (j = 0; j < C; j++) {
39             printf("matris[%d][%d]: ", i, j);
40             scanf("%d", &matris[i][j]);
41         }
42     }

```

```

43     // TRANSPOZU HESAPLAMA
44     // Satır ile sütun yer değiştirir: transpoz[j][i] = matris[i][j]
45     for (i = 0; i < R; i++) {
46         for (j = 0; j < C; j++) {
47             transpoz[j][i] = matris[i][j];
48         }
49     }
50
51     // ORIJİNAL MATRİSİ EKRANA YAZDIRMA
52     printf("\nOrijinal Matris:\n");
53
54     for (i = 0; i < R; i++) {
55         for (j = 0; j < C; j++) {
56             printf("%d ", matris[i][j]);
57         }
58         printf("\n"); // Her satırdan sonra alt satıra geç
59     }
60
61     // TRANSPOZ MATRİSİ EKRANA YAZDIRMA
62     printf("\nTranspoz Matris:\n");
63
64     for (i = 0; i < C; i++) {
65         for (j = 0; j < R; j++) {
66             printf("%d ", transpoz[i][j]);
67         }
68         printf("\n"); // Her satırdan sonra alt satıra geç
69     }
70
71     // AYRILAN BELLEĞİ SERBEST BIRAKMA
72     // Önce orijinal matrisin her satırı serbest bırakılır
73     for (i = 0; i < R; i++) {
74         free(matris[i]);
75     }
76     free(matris); // Sonra satır adreslerini tutan ana pointer serbest bırakılır
77
78     // Sonra transpoz matrisin her satırı serbest bırakılır
79     for (i = 0; i < C; i++) {
80         free(transpoz[i]);
81     }
82     free(transpoz); // En son transpoz ana pointerı serbest bırakılır
83
84     return 0;
85 }

```

Matrisin satir sayisini girin: 2
Matrisin sutun sayisini girin: 3
Matris elemanlarini girin:

matris[0][0]: 1
matris[0][1]: 9
matris[0][2]: 7
matris[1][0]: 5
matris[1][1]: 0
matris[1][2]: 3

Original Matris:

1 9 7
5 0 3

Transpoz Matris:

1 5
9 0
7 3